

Emporix Punchout Plugin — Implementation Plan

Emporix Punchout Plugin Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (– []) syntax for tracking.

Goal: Build a Node.js/TypeScript Emporix Marketplace plugin that enables cXML 1.2 and OCI 4.0 punchout for Emporix-powered supplier stores.

Architecture: Single Express service with Redis session store, deployed as a Docker container. Protocol handlers (cXML, OCI), Emporix API client, and admin UI extension are isolated internal modules with defined TypeScript interfaces between them.

Tech Stack: Node.js 20 LTS, TypeScript 5, Express 4, ioredis, fast-xml-parser, xmlbuilder2, axios, jsonwebtoken, jwks-rsa, bcryptjs, express-rate-limit, uuid, React 18 + Vite (admin UI), Jest + ts-jest +nock + ioredis-mock (tests)

Note: Emporix API paths below are based on Emporix’s documented REST API conventions. Verify each endpoint against the official Emporix API reference before implementation.

File Structure

src/	
index.ts	# Express app bootstrap and route
mounting	
config.ts	# Env var loading
crypto.ts	# AES-256-GCM encrypt/decrypt +
bcryptjs helpers	
session/	
types.ts	# PunchoutSession interface
store.ts	# Redis CRUD for tokens and
sessions	
protocols/	

cxml/	
types.ts	# CxmlSetupRequest, CxmlCartItem
interfaces	
parser.ts	# Parse raw XML → CxmlSetupRequest
generator.ts	# Build PunchOutSetupResponse +
PunchOutOrderMessage XML	
oci/	
types.ts	# OciSetupRequest, OciReturnItem
interfaces	
parser.ts	# Parse OCI form fields →
OciSetupRequest	
generator.ts	# Build indexed OCI return fields
emporix/	
types.ts	# EmporixCart, EmporixCartItem,
CustomerGroup	
auth.ts	# OAuth2 client credentials token
cache	
client.ts	# Cart, customer group API calls
middleware/	
rateLimiter.ts	# express-rate-limit for punchout
endpoints	
emporixJwt.ts	# Verify Emporix JWT via JWKS,
attach tenantId to req	
admin/	
configStore.ts	# PluginConfig read/write via
Emporix Config Service	
router.ts	# /admin/config, /admin/buyers
routes	
routes/	
punchout.ts	# /punchout/cxml/setup, /punchout/
oci/setup, /punchout/return	
session.ts	# GET /session/:token
widget.ts	# GET /punchout-widget.js
admin-ui/	
index.html	
vite.config.ts	
src/	
main.tsx	
App.tsx	# Tab nav, JWT extraction from URL
hash	
api.ts	# fetch wrapper that attaches JWT
auth header	
screens/	
Credentials.tsx	
BuyerMappings.tsx	
ProtocolSettings.tsx	
ConnectionTest.tsx	
tests/	
crypto.test.ts	
session/store.test.ts	
protocols/cxml/parser.test.ts	
protocols/cxml/generator.test.ts	

```
protocols/oci/parser.test.ts
protocols/oci/generator.test.ts
emporix/auth.test.ts
emporix/client.test.ts
middleware/emporixJwt.test.ts
routes/punchout.test.ts
routes/session.test.ts
routes/admin.test.ts
integration/punchout-flow.test.ts
```

```
plugin.json
Dockerfile
docker-compose.yml
package.json
tsconfig.json
jest.config.ts
```

Task 1: Project Scaffold

Files: - Create: package.json - Create: tsconfig.json - Create: jest.config.ts - Create: src/config.ts - Create: src/index.ts



Step 1: Create package.json

```
{
  "name": "emporix-punchout-plugin",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js",
    "dev": "ts-node-dev --respawn src/index.ts",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:integration": "jest --testPathPattern=integration"
  },
  "dependencies": {
    "axios": "^1.6.0",
    "bcryptjs": "^2.4.3",
    "express": "^4.18.2",
    "express-rate-limit": "^7.1.5",
    "fast-xml-parser": "^4.3.4",
    "ioredis": "^5.3.2",
    "jsonwebtoken": "^9.0.2",
    "jwks-rsa": "^3.1.0",
    "uuid": "^9.0.1",
    "xmlbuilder2": "^3.1.1"
  },
  "devDependencies": {
    "@types/bcryptjs": "^2.4.6",
```

```

    "@types/express": "^4.17.21",
    "@types/jest": "^29.5.11",
    "@types/jsonwebtoken": "^9.0.5",
    "@types/node": "^20.10.6",
    "@types/uuid": "^9.0.7",
    "ioredis-mock": "^8.9.0",
    "nock": "^13.4.0",
    "ts-jest": "^29.1.1",
    "ts-node-dev": "^2.0.0",
    "typescript": "^5.3.3"
  }
}

```



Step 2: Create tsconfig.json

```

{
  "compilerOptions": {
    "target": "ES2022",
    "module": "commonjs",
    "lib": ["ES2022"],
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "skipLibCheck": true
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist", "tests", "admin-ui"]
}

```



Step 3: Create jest.config.ts

```

import type { Config } from 'jest';

const config: Config = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  roots: ['<rootDir>/tests'],
  testMatch: ['**/*.test.ts'],
  moduleNameMapper: {
    '^ioredis$': '<rootDir>/node_modules/ioredis-mock',
  },
  testPathIgnorePatterns: ['/node_modules/', '/dist/'],
};

export default config;

```



Step 4: Create src/config.ts

```

export const config = {
  port: parseInt(process.env.PORT ?? '3000', 10),
  pluginHost: process.env.PLUGIN_HOST ?? 'http://localhost:3000',
  redis: {
    host: process.env.REDIS_HOST ?? 'localhost',
    port: parseInt(process.env.REDIS_PORT ?? '6379', 10),
  },
  emporix: {
    apiBase: process.env.EMPORIX_API_BASE ?? 'https://api.emporix.io',
    tenantId: process.env.EMPORIX_TENANT_ID ?? '',
    jwksUri: process.env.EMPORIX_JWKS_URI ?? 'https://api.emporix.io/.well-known/jwks.json',
  },
  crypto: {
    aesKey: process.env.AES_KEY ?? '',
  },
  outboundTimeoutMs: parseInt(process.env.OUTBOUND_TIMEOUT_MS ?? '5000', 10),
  tokenTtlSeconds: 15 * 60,
  sessionTtlSeconds: 2 * 60 * 60,
};

```



Step 5: Create src/index.ts

```

import express from 'express';
import { config } from './config';

const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.text({ type: 'text/xml' }));

app.get('/health', (_req, res) => res.json({ status: 'ok' }));

if (require.main === module) {
  app.listen(config.port, () => {
    console.log(`Punchout plugin listening on port ${config.port}`);
  });
}

export { app };

```



Step 6: Install dependencies and verify build

```

npm install
npx tsc --noEmit

```

Expected: no TypeScript errors.



Step 7: Commit

```
git init
git add package.json tsconfig.json jest.config.ts src/config.ts
      src/index.ts
git commit -m "feat: project scaffold – Express/TypeScript app
      with config"
```

Task 2: Session Types and Redis Store

Files: - Create: src/session/types.ts - Create: src/session/store.ts -
Create: tests/session/store.test.ts



Step 1: Create src/session/types.ts

```
export interface PunchoutSession {
  sessionId: string;
  protocol: 'cxml' | 'oci';
  buyerOrgId: string;
  customerGroupId: string;
  browserFormPostUrl: string;
  buyerCookie: string;
  emporixCartId: string | null;
  createdAt: string;
}
```



Step 2: Write failing tests — tests/session/store.test.ts

```
import { SessionStore } from '../../src/session/store';
import type { PunchoutSession } from '../../src/session/types';

const makeSession = (overrides: Partial<PunchoutSession> = {}):
  PunchoutSession => ({
  sessionId: 'sess-001',
  protocol: 'cxml',
  buyerOrgId: 'buyer-org-1',
  customerGroupId: 'cg-1',
  browserFormPostUrl: 'https://buyer.example.com/return',
  buyerCookie: 'cookie-abc',
  emporixCartId: null,
  createdAt: new Date().toISOString(),
  ...overrides,
});

describe('SessionStore', () => {
  let store: SessionStore;

  beforeEach(() => {
    store = new SessionStore({ host: 'localhost', port: 6379 });
```

```

});

afterEach(async () => {
  await store.disconnect();
});

it('saves and retrieves a session', async () => {
  const session = makeSession();
  await store.saveSession(session, 7200);
  const retrieved = await store.getSession('sess-001');
  expect(retrieved).toEqual(session);
});

it('returns null for missing session', async () => {
  const result = await store.getSession('nonexistent');
  expect(result).toBeNull();
});

it('saves a token mapping and consumes it once', async () => {
  await store.saveToken('token-xyz', 'sess-001', 900);
  const sessionId = await store.consumeToken('token-xyz');
  expect(sessionId).toBe('sess-001');
  const second = await store.consumeToken('token-xyz');
  expect(second).toBeNull();
});

it('returns null for expired/missing token', async () => {
  const result = await store.consumeToken('no-such-token');
  expect(result).toBeNull();
});

it('updates cart ID on an existing session', async () => {
  await store.saveSession(makeSession(), 7200);
  await store.updateCartId('sess-001', 'cart-999');
  const updated = await store.getSession('sess-001');
  expect(updated?.emporixCartId).toBe('cart-999');
});

it('deletes a session', async () => {
  await store.saveSession(makeSession(), 7200);
  await store.deleteSession('sess-001');
  const result = await store.getSession('sess-001');
  expect(result).toBeNull();
});
});

```



Step 3: Run to confirm failure

`npx jest tests/session/store.test.ts --no-coverage`

Expected: FAIL — SessionStore not found.



Step 4: Create src/session/store.ts

```
import Redis from 'ioredis';
import type { PunchoutSession } from './types';

const TOKEN_PREFIX = 'punchout:session:';
const ACTIVE_PREFIX = 'punchout:active:';

export class SessionStore {
  private redis: Redis;

  constructor(opts: { host: string; port: number }) {
    this.redis = new Redis({ host: opts.host, port: opts.port,
      lazyConnect: true });
  }

  async saveToken(token: string, sessionId: string, ttlSeconds:
    number): Promise<void> {
    await this.redis.set(`${TOKEN_PREFIX}${token}`, sessionId,
      'EX', ttlSeconds);
  }

  async consumeToken(token: string): Promise<string | null> {
    const key = `${TOKEN_PREFIX}${token}`;
    const [sessionId] = await
      this.redis.multi().get(key).del(key).exec() as [string |
        null, number][];
    return sessionId ?? null;
  }

  async saveSession(session: PunchoutSession, ttlSeconds:
    number): Promise<void> {
    await this.redis.set(
      `${ACTIVE_PREFIX}${session.sessionId}`,
      JSON.stringify(session),
      'EX',
      ttlSeconds,
    );
  }

  async getSession(sessionId: string): Promise<PunchoutSession |
    null> {
    const raw = await this.redis.get(`${ACTIVE_PREFIX}${sessionId}`);
    if (!raw) return null;
    return JSON.parse(raw) as PunchoutSession;
  }

  async updateCartId(sessionId: string, cartId: string):
    Promise<void> {
    const session = await this.getSession(sessionId);
    if (!session) return;
  }
}
```



```

    const ttl = await this.redis.ttl(`${ACTIVE_PREFIX}${sessionId}`);
    session.emporixCartId = cartId;
    await this.redis.set(`${ACTIVE_PREFIX}${sessionId}`,
        JSON.stringify(session), 'EX', ttl > 0 ? ttl : 7200);
  }

  async deleteSession(sessionId: string): Promise<void> {
    await this.redis.del(`${ACTIVE_PREFIX}${sessionId}`);
  }

  async disconnect(): Promise<void> {
    await this.redis.quit();
  }
}

```



Step 5: Run tests and confirm pass

```
npx jest tests/session/store.test.ts --no-coverage
```

Expected: 6 passing.



Step 6: Commit

```

git add src/session/ tests/session/
git commit -m "feat: session types and Redis store"

```

Task 3: Crypto Utilities

Files: - Create: src/crypto.ts - Create: tests/crypto.test.ts



Step 1: Write failing tests — tests/crypto.test.ts

```

import { encrypt, decrypt, hashSecret, verifySecret } from '../../src/crypto';

const KEY = 'a'.repeat(32); // 32-byte key for AES-256

describe('encrypt/decrypt', () => {
  it('round-trips a plaintext string', () => {
    const cipher = encrypt('hello world', KEY);
    expect(decrypt(cipher, KEY)).toBe('hello world');
  });

  it('produces different ciphertext each call (unique IV)', () => {
    {
      const a = encrypt('same', KEY);
      const b = encrypt('same', KEY);
    }
  });
}

```

```

    expect(a).not.toBe(b);
  });

  it('throws on tampered ciphertext', () => {
    const cipher = encrypt('secret', KEY);
    const parts = cipher.split(':');
    parts[2] = Buffer.from('tampered').toString('base64');
    expect(() => decrypt(parts.join(':'), KEY)).toThrow();
  });
});

describe('hashSecret / verifySecret', () => {
  it('verifies a correct secret against its hash', async () => {
    const hash = await hashSecret('mysecret');
    expect(await verifySecret('mysecret', hash)).toBe(true);
  });

  it('rejects a wrong secret', async () => {
    const hash = await hashSecret('mysecret');
    expect(await verifySecret('wrong', hash)).toBe(false);
  });
});

```



Step 2: Run to confirm failure

```
npx jest tests/crypto.test.ts --no-coverage
```

Expected: FAIL — encrypt not found.



Step 3: Create src/crypto.ts

```

import { createCipheriv, createDecipheriv, randomBytes } from
  'crypto';
import bcrypt from 'bcryptjs';

const ALGORITHM = 'aes-256-gcm';
const IV_LENGTH = 12;
const SALT_ROUNDS = 10;

export function encrypt(plaintext: string, key: string): string {
  const iv = randomBytes(IV_LENGTH);
  const cipher = createCipheriv(ALGORITHM, Buffer.from(key), iv);
  const encrypted = Buffer.concat([cipher.update(plaintext,
    'utf8'), cipher.final()]);
  const authTag = cipher.getAuthTag();
  return `${iv.toString('base64')}:${authTag.toString('base64')}:${
    encrypted.toString('base64')}`;
}

export function decrypt(ciphertext: string, key: string): string {
  const [ivB64, authTagB64, dataB64] = ciphertext.split(':');

```

```

    const decipher = createDecipheriv(ALGORITHM, Buffer.from(key),
        Buffer.from(ivB64, 'base64'));
    decipher.setAuthTag(Buffer.from(authTagB64, 'base64'));
    return Buffer.concat([
        decipher.update(Buffer.from(dataB64, 'base64')),
        decipher.final(),
    ]).toString('utf8');
}

export async function hashSecret(secret: string): Promise<string>
{
    return bcrypt.hash(secret, SALT_ROUNDS);
}

export async function verifySecret(secret: string, hash: string):
    Promise<boolean> {
    return bcrypt.compare(secret, hash);
}

```



Step 4: Run tests and confirm pass

```
npx jest tests/crypto.test.ts --no-coverage
```

Expected: 5 passing.



Step 5: Commit

```

git add src/crypto.ts tests/crypto.test.ts
git commit -m "feat: AES-256-GCM encrypt/decrypt and bcrypt
    helpers"

```

Task 4: cXML Parser

Files: - Create: src/protocols/cxml/types.ts - Create: src/protocols/cxml/parser.ts - Create: tests/protocols/cxml/parser.test.ts



Step 1: Create src/protocols/cxml/types.ts

```

export interface CxmlSetupRequest {
    payloadId: string;
    buyerOrgId: string;
    sharedSecret: string;
    buyerCookie: string;
    browserFormPostUrl: string;
    operation: 'create' | 'edit' | 'inspect';
}

export interface CxmlCartItem {

```

```

    sku: string;
    name: string;
    quantity: number;
    unitPrice: number;
    currency: string;
    uom: string;
  }

```



Step 2: Write failing tests — tests/protocols/cxml/parser.test.ts

```

import { parsePunchOutSetupRequest } from '../../src/protocols/
  cxml/parser';

```

```

const VALID_XML = `<?xml version="1.0" encoding="UTF-8"?>
<cXML payloadID="test-payload-1" timestamp="2026-05-11T10:00:00Z">
  <Header>
    <From>
      <Credential domain="NetworkId">
        <Identity>buyer-org-001</Identity>
      </Credential>
    </From>
    <To>
      <Credential domain="NetworkId">
        <Identity>supplier@emporix.com</Identity>
      </Credential>
    </To>
    <Sender>
      <Credential domain="NetworkId">
        <Identity>buyer-org-001</Identity>
        <SharedSecret>s3cr3t</SharedSecret>
      </Credential>
      <UserAgent>Ariba Buyer 8.2</UserAgent>
    </Sender>
  </Header>
  <Request deploymentMode="production">
    <PunchOutSetupRequest operation="create">
      <BuyerCookie>cookie-abc-123</BuyerCookie>
      <BrowserFormPost>
        <URL>https://buyer.example.com/cxml/return</URL>
      </BrowserFormPost>
    </PunchOutSetupRequest>
  </Request>
</cXML>`;

```

```

describe('parsePunchOutSetupRequest', () => {
  it('parses a valid cXML setup request', () => {
    const result = parsePunchOutSetupRequest(VALID_XML);
    expect(result.payloadId).toBe('test-payload-1');
    expect(result.buyerOrgId).toBe('buyer-org-001');
    expect(result.sharedSecret).toBe('s3cr3t');
    expect(result.buyerCookie).toBe('cookie-abc-123');
  });
});

```

```

    expect(result.browserFormPostUrl).toBe('https://
      buyer.example.com/cxml/return');
    expect(result.operation).toBe('create');
  });

  it('throws on malformed XML', () => {
    expect(() =>
      parsePunchOutSetupRequest('<broken>')).toThrow('Invalid
      cXML');
  });

  it('throws when BuyerCookie is missing', () => {
    const xml = VALID_XML.replace('<BuyerCookie>cookie-abc-123</
      BuyerCookie>', '');
    expect(() => parsePunchOutSetupRequest(xml)).toThrow('Missing
      BuyerCookie');
  });

  it('throws when BrowserFormPost URL is missing', () => {
    const xml = VALID_XML.replace(
      '<BrowserFormPost>\n      <URL>https://buyer.example.com/
      cxml/return</URL>\n      </BrowserFormPost>',
      ''
    );
    expect(() => parsePunchOutSetupRequest(xml)).toThrow('Missing
      BrowserFormPost URL');
  });

  it('throws when SharedSecret is missing', () => {
    const xml = VALID_XML.replace('<SharedSecret>s3cr3t</
      SharedSecret>', '');
    expect(() => parsePunchOutSetupRequest(xml)).toThrow('Missing
      SharedSecret');
  });
});

```



Step 3: Run to confirm failure

```
npx jest tests/protocols/cxml/parser.test.ts --no-coverage
```

Expected: FAIL — parsePunchOutSetupRequest not found.



Step 4: Create src/protocols/cxml/parser.ts

```

import { XMLParser } from 'fast-xml-parser';
import type { CxmlSetupRequest } from './types';

const parser = new XMLParser({ ignoreAttributes: false,
  attributeNamePrefix: '@_' });

export function parsePunchOutSetupRequest(xml: string):
  CxmlSetupRequest {

```

```

let doc: Record<string, unknown>;
try {
  doc = parser.parse(xml) as Record<string, unknown>;
} catch {
  throw new Error('Invalid cXML: parse error');
}

const root = doc['cXML'] as Record<string, unknown> | undefined;
if (!root) throw new Error('Invalid cXML: missing root
  element');

const payloadId = String(root['@payloadID'] ?? '');

const header = root['Header'] as Record<string, unknown> |
  undefined;
if (!header) throw new Error('Invalid cXML: missing Header');

const from = header['From'] as Record<string, unknown>;
const buyerOrgId = String(
  (from?.['Credential'] as Record<string, unknown>)?.
    ['Identity'] ?? ''
);

const sender = header['Sender'] as Record<string, unknown>;
const senderCred = sender?.['Credential'] as Record<string,
  unknown>;
const sharedSecret = String(senderCred?.['SharedSecret'] ?? '');
if (!sharedSecret) throw new Error('Missing SharedSecret');

const request = root['Request'] as Record<string, unknown>;
const setupReq = request?.['PunchOutSetupRequest'] as
  Record<string, unknown>;
if (!setupReq) throw new Error('Invalid cXML: missing
  PunchOutSetupRequest');

const buyerCookie = String(setupReq['BuyerCookie'] ?? '');
if (!buyerCookie) throw new Error('Missing BuyerCookie');

const bfp = setupReq['BrowserFormPost'] as Record<string,
  unknown>;
const browserFormPostUrl = String(bfp?.['URL'] ?? '');
if (!browserFormPostUrl) throw new Error('Missing
  BrowserFormPost URL');

const operation = String((setupReq['@operation'] as string) ??
  'create') as CxmlSetupRequest['operation'];

return { payloadId, buyerOrgId, sharedSecret, buyerCookie,
  browserFormPostUrl, operation };
}

```



Step 5: Run tests and confirm pass

```
npx jest tests/protocols/cxml/parser.test.ts --no-coverage
```

Expected: 5 passing.



Step 6: Commit

```
git add src/protocols/cxml/ tests/protocols/cxml/parser.test.ts
git commit -m "feat: cXML PunchOutSetupRequest parser"
```

Task 5: cXML Generator

Files: - Create: src/protocols/cxml/generator.ts - Create: tests/protocols/cxml/generator.test.ts



Step 1: Write failing tests — tests/protocols/cxml/generator.test.ts

```
import { buildSetupResponse, buildOrderMessage } from '../../../src/protocols/cxml/generator';
import type { CxmlCartItem } from '../../../src/protocols/cxml/types';
import { XMLParser } from 'fast-xml-parser';

const xmlParser = new XMLParser({ ignoreAttributes: false,
  attributeNamePrefix: '@_' });

describe('buildSetupResponse', () => {
  it('returns valid cXML with StartPage URL', () => {
    const xml = buildSetupResponse('https://plugin.example.com/session/token-abc');
    const doc = xmlParser.parse(xml) as Record<string, unknown>;
    const cxml = doc['cXML'] as Record<string, unknown>;
    const response = cxml['Response'] as Record<string, unknown>;
    const status = response['Status'] as Record<string, unknown>;
    expect(status['@_code']).toBe('200');
    const startPage = (response['PunchOutSetupResponse'] as Record<string, unknown>)[
      'StartPage'] as Record<string, unknown>;
    expect(startPage['URL']).toBe('https://plugin.example.com/session/token-abc');
  });
});

describe('buildOrderMessage', () => {
  const items: CxmlCartItem[] = [
    { sku: 'SKU-001', name: 'Widget A', quantity: 2, unitPrice: 49.99, currency: 'USD', uom: 'EA' },
    { sku: 'SKU-002', name: 'Widget B', quantity: 1, unitPrice: 9.99, currency: 'USD', uom: 'EA' },
  ];
```

```

it('returns valid cXML PunchOutOrderMessage', () => {
  const xml = buildOrderMessage('cookie-abc', 'edit', items,
    'USD');
  const doc = xmlParser.parse(xml) as Record<string, unknown>;
  const cxml = doc['cXML'] as Record<string, unknown>;
  const message = cxml['Message'] as Record<string, unknown>;
  const poom = message['PunchOutOrderMessage'] as
    Record<string, unknown>;
  expect(poom['BuyerCookie']).toBe('cookie-abc');
});

it('includes all line items', () => {
  const xml = buildOrderMessage('cookie-abc', 'edit', items,
    'USD');
  const doc = xmlParser.parse(xml) as Record<string, unknown>;
  const poom = ((doc['cXML'] as Record<string, unknown>)
    ['Message'] as Record<string, unknown>)
    ['PunchOutOrderMessage'] as Record<string, unknown>;
  const itemsIn = Array.isArray(poom['ItemIn']) ?
    poom['ItemIn'] : [poom['ItemIn']];
  expect(itemsIn).toHaveLength(2);
});

it('handles empty cart', () => {
  const xml = buildOrderMessage('cookie-abc', 'edit', [],
    'USD');
  expect(xml).toContain('PunchOutOrderMessage');
});

it('escapes special characters in product names', () => {
  const specialItems: CxmlCartItem[] = [
    { sku: 'SKU-003', name: 'Widget <A&B>', quantity: 1,
      unitPrice: 5.00, currency: 'USD', uom: 'EA' },
  ];
  const xml = buildOrderMessage('cookie', 'edit', specialItems,
    'USD');
  expect(xml).not.toContain('Widget <A&B>');
  expect(xml).toContain('Widget');
});
});

```



Step 2: Run to confirm failure

`npx jest tests/protocols/cxml/generator.test.ts --no-coverage`

Expected: FAIL — buildSetupResponse not found.



Step 3: Create src/protocols/cxml/generator.ts

```

import { create } from 'xmlbuilder2';
import type { CxmlCartItem } from './types';

```



```

const CXML_DOCTYPE = 'http://xml.cxml.org/schemas/cXML/1.2.014/
    cXML.dtd';

function cxmlRoot(payloadId: string) {
    return create({ version: '1.0', encoding:
        'UTF-8' }).ele('cXML', {
        payloadID: payloadId,
        timestamp: new Date().toISOString(),
        'xmlns:xsi': 'http://www.w3.org/2001/XMLSchema-instance',
    });
}

export function buildSetupResponse(startPageUrl: string): string {
    const payloadId = `${Date.now()}@emporix-punchout-plugin`;
    const root = cxmlRoot(payloadId)
        .ele('Response')
        .ele('Status', { code: '200', text: 'OK' }).up()
        .ele('PunchOutSetupResponse')
        .ele('StartPage')
        .ele('URL').txt(startPageUrl).up()
        .up()
        .up()
        .up();
    return root.end({ prettyPrint: false });
}

export function buildOrderMessage(
    buyerCookie: string,
    operationAllowed: string,
    items: CxmlCartItem[],
    currency: string,
): string {
    const payloadId = `${Date.now()}@emporix-punchout-plugin`;
    const total = items.reduce((sum, i) => sum + i.unitPrice *
        i.quantity, 0).toFixed(2);

    const root = cxmlRoot(payloadId).ele('Message');
    const poom = root.ele('PunchOutOrderMessage');
    poom.ele('BuyerCookie').txt(buyerCookie).up();
    poom
        .ele('PunchOutOrderMessageHeader', { operationAllowed })
        .ele('Total')
        .ele('Money', { currency }).txt(total).up()
        .up()
        .up();

    for (const item of items) {
        poom
            .ele('ItemIn', { quantity: String(item.quantity) })
            .ele('ItemID')
            .ele('SupplierPartID').txt(item.sku).up()
            .up()
            .ele('ItemDetail')

```

```

        .ele('UnitPrice').ele('Money', { currency:
            item.currency }).txt(item.unitPrice.toFixed(2)).up().up()
        .ele('Description', { 'xml:lang':
            'en' }).txt(item.name).up()
        .ele('UnitOfMeasure').txt(item.uom).up()
        .up()
        .up();
    }

    return root.end({ prettyPrint: false });
}

```



Step 4: Run tests and confirm pass

```
npx jest tests/protocols/cxml/generator.test.ts --no-coverage
```

Expected: 4 passing.



Step 5: Commit

```

git add src/protocols/cxml/generator.ts tests/protocols/cxml/
generator.test.ts
git commit -m "feat: cXML PunchOutSetupResponse and
PunchOutOrderMessage generators"

```

Task 6: OCI Parser and Generator

Files: - Create: src/protocols/oci/types.ts - Create: src/protocols/oci/parser.ts - Create: src/protocols/oci/generator.ts - Create: tests/protocols/oci/parser.test.ts - Create: tests/protocols/oci/generator.test.ts



Step 1: Create src/protocols/oci/types.ts

```

export interface OciSetupRequest {
    hookUrl: string;
    username: string;
    password: string;
    okCode: string;
    caller: string;
}

export interface OciReturnItem {
    description: string;
    matnr: string;
    quantity: number;
    unit: string;
    price: number;
}

```

```

    currency: string;
    vendorMat: string;
}

```



Step 2: Write failing parser tests — tests/protocols/oci/parser.test.ts

```

import { parseOciSetupRequest } from '../../../src/protocols/oci/
  parser';

describe('parseOciSetupRequest', () => {
  it('parses valid OCI form fields', () => {
    const body = {
      HOOK_URL: 'https://sap.example.com/oci/return',
      USERNAME: 'buyer-org-002',
      PASSWORD: 'oci-secret',
      '~OkCode': 'ADDFROMCATALOG',
      '~Caller': 'SAPLMOB',
    };
    const result = parseOciSetupRequest(body);
    expect(result.hookUrl).toBe('https://sap.example.com/oci/
      return');
    expect(result.username).toBe('buyer-org-002');
    expect(result.password).toBe('oci-secret');
    expect(result.okCode).toBe('ADDFROMCATALOG');
    expect(result.caller).toBe('SAPLMOB');
  });

  it('throws when HOOK_URL is missing', () => {
    expect(() => parseOciSetupRequest({ USERNAME: 'x', PASSWORD:
      'y' })).toThrow('Missing HOOK_URL');
  });

  it('defaults okCode to ADDFROMCATALOG when absent', () => {
    const body = { HOOK_URL: 'https://sap.example.com/oci/
      return', USERNAME: 'x', PASSWORD: 'y' };
    const result = parseOciSetupRequest(body);
    expect(result.okCode).toBe('ADDFROMCATALOG');
  });
});

```



Step 3: Write failing generator tests — tests/protocols/oci/generator.test.ts

```

import { buildOciReturnFields } from '../../../src/protocols/oci/
  generator';
import type { OciReturnItem } from '../../../src/protocols/oci/
  types';

const items: OciReturnItem[] = [
  { description: 'Widget A', matnr: 'SKU-001', quantity: 2, unit:
    'EA', price: 49.99, currency: 'USD', vendorMat:
    'VM-001' },

```

```

    { description: 'Widget B', matnr: 'SKU-002', quantity: 1, unit:
      'EA', price: 9.99, currency: 'USD', vendorMat: 'VM-002' },
  ];

describe('buildOciReturnFields', () => {
  it('generates indexed fields for all items', () => {
    const fields = buildOciReturnFields(items, 'ADDFROMCATALOG');
    expect(fields['NEW_ITEM-DESCRIPTION[1]']).toBe('Widget A');
    expect(fields['NEW_ITEM-MATNR[1]']).toBe('SKU-001');
    expect(fields['NEW_ITEM-QUANTITY[1]']).toBe('2');
    expect(fields['NEW_ITEM-DESCRIPTION[2]']).toBe('Widget B');
    expect(fields['~OkCode']).toBe('ADDFROMCATALOG');
  });

  it('handles empty cart', () => {
    const fields = buildOciReturnFields([], 'ADDFROMCATALOG');
    expect(fields['~OkCode']).toBe('ADDFROMCATALOG');
    expect(Object.keys(fields).filter(k =>
      k.startsWith('NEW_ITEM'))).toHaveLength(0);
  });
});

```



Step 4: Run to confirm failures

```
npx jest tests/protocols/oci/ --no-coverage
```

Expected: FAIL — modules not found.



Step 5: Create src/protocols/oci/parser.ts

```

import type { OciSetupRequest } from './types';

export function parseOciSetupRequest(body: Record<string,
  string>): OciSetupRequest {
  const hookUrl = body['HOOK_URL'];
  if (!hookUrl) throw new Error('Missing HOOK_URL');
  return {
    hookUrl,
    username: body['USERNAME'] ?? '',
    password: body['PASSWORD'] ?? '',
    okCode: body['~OkCode'] ?? 'ADDFROMCATALOG',
    caller: body['~Caller'] ?? '',
  };
}

```



Step 6: Create src/protocols/oci/generator.ts

```

import type { OciReturnItem } from './types';

export function buildOciReturnFields(

```

```

    items: OciReturnItem[],
    okCode: string,
  ): Record<string, string> {
    const fields: Record<string, string> = { '~OkCode': okCode };
    items.forEach((item, idx) => {
      const n = idx + 1;
      fields[`NEW_ITEM-DESCRIPTION[${n}]`] = item.description;
      fields[`NEW_ITEM-MATNR[${n}]`] = item.matnr;
      fields[`NEW_ITEM-QUANTITY[${n}]`] = String(item.quantity);
      fields[`NEW_ITEM-UNIT[${n}]`] = item.unit;
      fields[`NEW_ITEM-PRICE[${n}]`] = item.price.toFixed(2);
      fields[`NEW_ITEM-CURRENCY[${n}]`] = item.currency;
      fields[`NEW_ITEM-VENDMAT[${n}]`] = item.vendorMat;
    });
    return fields;
  }
}

```



Step 7: Run tests and confirm pass

```
npx jest tests/protocols/oci/ --no-coverage
```

Expected: 5 passing.



Step 8: Commit

```

git add src/protocols/oci/ tests/protocols/oci/
git commit -m "feat: OCI 4.0 parser and return field generator"

```

Task 7: Emporix Auth Client

Files: - Create: src/emporix/types.ts - Create: src/emporix/auth.ts - Create: tests/emporix/auth.test.ts



Step 1: Create src/emporix/types.ts

```

export interface EmporixCartItem {
  itemId: string;
  sku: string;
  name: string;
  quantity: number;
  price: { amount: number; currency: string };
  uom: string;
}

export interface EmporixCart {
  cartId: string;
  items: EmporixCartItem[];
  currency: string;
}

```

```
}
```

```
export interface CustomerGroup {  
  id: string;  
  name: string;  
}
```



Step 2: Write failing tests — tests/emporix/auth.test.ts

```
import nock from 'nock';  
import { TokenCache } from '../../src/emporix/auth';  
  
const API_BASE = 'https://api.emporix.io';  
const TENANT = 'test-tenant';  
  
describe('TokenCache', () => {  
  afterEach(() => nock.cleanAll());  
  
  it('fetches a token on first call', async () => {  
    nock(API_BASE)  
      .post(`/customerlogin/auth/anonymous/token`)  
      .reply(200, { access_token: 'tok-abc', expires_in: 3600,  
        token_type: 'Bearer' });  
  
    const cache = new TokenCache(API_BASE, TENANT, 'client-id',  
      'client-secret');  
    const token = await cache.getToken();  
    expect(token).toBe('tok-abc');  
  });  
  
  it('reuses cached token on second call', async () => {  
    nock(API_BASE)  
      .post(`/customerlogin/auth/anonymous/token`)  
      .once()  
      .reply(200, { access_token: 'tok-abc', expires_in: 3600,  
        token_type: 'Bearer' });  
  
    const cache = new TokenCache(API_BASE, TENANT, 'client-id',  
      'client-secret');  
    await cache.getToken();  
    const second = await cache.getToken();  
    expect(second).toBe('tok-abc');  
    expect(nock.pendingMocks()).toHaveLength(0);  
  });  
  
  it('throws on auth failure', async () => {  
    nock(API_BASE)  
      .post(`/customerlogin/auth/anonymous/token`)  
      .reply(401, { error: 'invalid_client' });  
  
    const cache = new TokenCache(API_BASE, TENANT, 'bad-id', 'bad-  
      secret');
```

```

    await expect(cache.getToken()).rejects.toThrow('Emporix auth
    failed');
  });
});

```



Step 3: Run to confirm failure

```
npx jest tests/emporix/auth.test.ts --no-coverage
```

Expected: FAIL — TokenCache not found.



Step 4: Create src/emporix/auth.ts

```

import axios from 'axios';

export class TokenCache {
  private token: string | null = null;
  private expiresAt = 0;

  constructor(
    private apiBase: string,
    private tenantId: string,
    private clientId: string,
    private clientSecret: string,
  ) {}

  async getToken(): Promise<string> {
    if (this.token && Date.now() < this.expiresAt) return
      this.token;

    const params = new URLSearchParams({
      grant_type: 'client_credentials',
      client_id: this.clientId,
      client_secret: this.clientSecret,
    });

    let data: { access_token: string; expires_in: number };
    try {
      const res = await axios.post<typeof data>(
        `${this.apiBase}/customerlogin/auth/anonymous/token`,
        params.toString(),
        { headers: { 'Content-Type': 'application/x-www-form-
          urlencoded' }, timeout: 5000 },
      );
      data = res.data;
    } catch {
      throw new Error('Emporix auth failed: unable to obtain
        access token');
    }

    this.token = data.access_token;
    this.expiresAt = Date.now() + (data.expires_in - 30) * 1000;
  }
}

```

```
    return this.token;
  }
}
```



Step 5: Run tests and confirm pass

```
npx jest tests/emporix/auth.test.ts --no-coverage
```

Expected: 3 passing.



Step 6: Commit

```
git add src/emporix/types.ts src/emporix/auth.ts tests/emporix/
auth.test.ts
git commit -m "feat: Emporix OAuth2 token cache"
```

Task 8: Emporix API Client

Files: - Create: src/emporix/client.ts - Create: tests/emporix/client.test.ts



Step 1: Write failing tests — tests/emporix/client.test.ts

```
import nock from 'nock';
import { EmporixClient } from '../../src/emporix/client';

const API_BASE = 'https://api.emporix.io';
const TENANT = 'test-tenant';
const TOKEN = 'Bearer test-token';

describe('EmporixClient', () => {
  let client: EmporixClient;

  beforeEach(() => {
    client = new EmporixClient(API_BASE, TENANT, 5000);
  });

  afterEach(() => nock.cleanAll());

  describe('createGuestCart', () => {
    it('creates a cart and returns cartId', async () => {
      nock(API_BASE)
        .post(`/cart/${TENANT}/carts`)
        .reply(201, { cartId: 'cart-001', currency: 'USD' });

      const cartId = await client.createGuestCart('cg-premium',
        TOKEN);
      expect(cartId).toBe('cart-001');
    });
  });
});
```



```

});

it('throws on Emporix error', async () => {
  nock(API_BASE).post(`/cart/${TENANT}/carts`).reply(500, {});
  await expect(client.createGuestCart('cg-premium',
    TOKEN)).rejects.toThrow('Failed to create Emporix cart');
});

describe('getCart', () => {
  it('returns cart with items', async () => {
    nock(API_BASE)
      .get(`/cart/${TENANT}/carts/cart-001`)
      .reply(200, {
        cartId: 'cart-001',
        currency: 'USD',
        items: [
          { itemId: 'i1', sku: 'SKU-001', name: 'Widget',
            quantity: 2, price: { amount: 49.99, currency: 'USD' },
            uom: 'EA' },
        ],
      });

    const cart = await client.getCart('cart-001', TOKEN);
    expect(cart.cartId).toBe('cart-001');
    expect(cart.items).toHaveLength(1);
    expect(cart.items[0].sku).toBe('SKU-001');
  });
});

describe('getCustomerGroups', () => {
  it('returns list of customer groups', async () => {
    nock(API_BASE)
      .get(`/customer-group/${TENANT}/customergroups`)
      .reply(200, [{ id: 'cg-1', name: 'Premium' }, { id:
        'cg-2', name: 'Standard' }]);

    const groups = await client.getCustomerGroups(TOKEN);
    expect(groups).toHaveLength(2);
    expect(groups[0].id).toBe('cg-1');
  });
});
});

```



Step 2: Run to confirm failure

`npx jest tests/emporix/client.test.ts --no-coverage`

Expected: FAIL — EmporixClient not found.



Step 3: Create src/emporix/client.ts

```
import axios from 'axios';
import type { EmporixCart, CustomerGroup } from './types';

export class EmporixClient {
  constructor(
    private apiBase: string,
    private tenantId: string,
    private timeoutMs: number,
  ) {}

  private get axiosOpts() {
    return { timeout: this.timeoutMs };
  }

  async createGuestCart(customerGroupId: string, accessToken:
    string): Promise<string> {
    try {
      const res = await axios.post<{ cartId: string }>(
        `${this.apiBase}/cart/${this.tenantId}/carts`,
        { currency: 'USD', customerGroup: customerGroupId },
        { ...this.axiosOpts, headers: { Authorization:
          accessToken } } },
      );
      return res.data.cartId;
    } catch {
      throw new Error('Failed to create Emporix cart');
    }
  }

  async getCart(cartId: string, accessToken: string):
    Promise<EmporixCart> {
    const res = await axios.get<EmporixCart>(
      `${this.apiBase}/cart/${this.tenantId}/carts/${cartId}`,
      { ...this.axiosOpts, headers: { Authorization:
        accessToken } } },
    );
    return res.data;
  }

  async getCustomerGroups(accessToken: string):
    Promise<CustomerGroup[]> {
    const res = await axios.get<CustomerGroup[]>(
      `${this.apiBase}/customer-group/${this.tenantId}/
        customergroups`,
      { ...this.axiosOpts, headers: { Authorization:
        accessToken } } },
    );
    return res.data;
  }
}
```



Step 4: Run tests and confirm pass

```
npx jest tests/emporix/client.test.ts --no-coverage
```

Expected: 4 passing.



Step 5: Commit

```
git add src/emporix/client.ts tests/emporix/client.test.ts
git commit -m "feat: Emporix cart and customer group API client"
```

Task 9: Admin Config Store

Files: - Create: src/admin/configStore.ts



Step 1: Create src/admin/configStore.ts

The PluginConfig type and CRUD operations backed by the Emporix Configuration Service.

```
import axios from 'axios';
import { encrypt, decrypt, hashSecret } from '../crypto';
import { config as appConfig } from '../config';

export interface PluginConfig {
  sharedSecretHash: string;
  serviceAccount: { clientId: string; clientSecret: string };
  storefrontBaseUrl: string;
  cxmlEnabled: boolean;
  ociEnabled: boolean;
  operationAllowed: 'create' | 'edit' | 'inspect';
  ociOkCode: 'ADDFROMCATALOG' | 'SOURCING';
  buyerMappings: Array<{ buyerOrgId: string; customerGroupId:
    string }>;
}

const CONFIG_KEY = 'punchout-plugin';

export async function getConfig(tenantId: string, accessToken:
  string): Promise<PluginConfig | null> {
  try {
    const res = await axios.get<{ value: PluginConfig }>(
      `${appConfig.emporix.apiBase}/configuration/${tenantId}/
      configurations/${CONFIG_KEY}`,
      { headers: { Authorization: `Bearer ${accessToken}` },
        timeout: 5000 },
    );
    const cfg = res.data.value;
```

```

    cfg.serviceAccount.clientSecret =
      decrypt(cfg.serviceAccount.clientSecret,
        appConfig.crypto.aesKey);
    return cfg;
  } catch (err: unknown) {
    if (axios.isAxiosError(err) && err.response?.status === 404)
      return null;
    throw err;
  }
}

export async function saveConfig(
  tenantId: string,
  incoming: Omit<PluginConfig, 'sharedSecretHash'> &
    { sharedSecret?: string },
  accessToken: string,
): Promise<void> {
  const existing = await getConfig(tenantId, accessToken);
  const sharedSecretHash = incoming.sharedSecret
    ? await hashSecret(incoming.sharedSecret)
    : (existing?.sharedSecretHash ?? '');

  const toSave: PluginConfig = {
    ...incoming,
    sharedSecretHash,
    serviceAccount: {
      clientId: incoming.serviceAccount.clientId,
      clientSecret: encrypt(incoming.serviceAccount.clientSecret,
        appConfig.crypto.aesKey),
    },
  };

  await axios.put(
    `${appConfig.emporix.apiBase}/configuration/${tenantId}/
      configurations/${CONFIG_KEY}`,
    { value: toSave },
    { headers: { Authorization: `Bearer ${accessToken}` },
      timeout: 5000 },
  );
}

```



Step 2: Commit

```

git add src/admin/configStore.ts
git commit -m
  "feat: admin config store with AES-encrypted service
  account credentials"

```

Task 10: Middleware — Rate Limiter and Emporix JWT

Files: - Create: src/middleware/rateLimiter.ts - Create: src/middleware/emporixJwt.ts - Create: tests/middleware/emporixJwt.test.ts



Step 1: Create src/middleware/rateLimiter.ts

```
import rateLimit from 'express-rate-limit';

export const punchoutRateLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 15,
  standardHeaders: true,
  legacyHeaders: false,
  message: 'Too many punchout requests, please try again later.',
});
```



Step 2: Write failing JWT middleware tests — tests/middleware/emporixJwt.test.ts

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { emporixJwtMiddleware } from '../../src/middleware/emporixJwt';

jest.mock('jwks-rsa', () => ({
  expressJwtSecret: jest.fn(() => (_req: unknown, _header:
    unknown, _payload: unknown, cb: (err: null, secret:
    string) => void) => {
    cb(null, 'test-secret');
  }),
})),
}));

const makeReq = (token?: string) => ({
  headers: token ? { authorization: `Bearer ${token}` } : {},
} as unknown as Request);

const makeRes = () => {
  const res = { status: jest.fn().mockReturnThis(), json:
    jest.fn() } as unknown as Response;
  return res;
};

describe('emporixJwtMiddleware', () => {
  const next = jest.fn() as NextFunction;

  it('calls next() with a valid token', async () => {
    const token = jwt.sign({ tenantId: 'my-tenant', sub:
      'user-1' }, 'test-secret');
    const req = makeReq(token);
```

```

    const res = makeRes();
    await emporixJwtMiddleware(req, res, next);
    expect(next).toHaveBeenCalled();
  });

  it('returns 401 with no token', async () => {
    const req = makeReq();
    const res = makeRes();
    await emporixJwtMiddleware(req, res, next);
    expect(res.status).toHaveBeenCalledWith(401);
  });
});

```



Step 3: Run to confirm failure

```
npx jest tests/middleware/emporixJwt.test.ts --no-coverage
```

Expected: FAIL — emporixJwtMiddleware not found.



Step 4: Create src/middleware/emporixJwt.ts

```

import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import jwksRsa from 'jwks-rsa';
import { config } from '../config';

const client = jwksRsa({ jwksUri: config.emporix.jwksUri, cache:
  true, rateLimit: true });

function getKey(header: jwt.JwtHeader, callback:
  jwt.SigningKeyCallback) {
  client.getSigningKey(header.kid, (err, key) => {
    if (err) return callback(err);
    callback(null, key?.getPublicKey());
  });
}

export function emporixJwtMiddleware(req: Request, res: Response,
  next: NextFunction): void {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    res.status(401).json({ error: 'Missing authorization
      token' });
    return;
  }

  const token = authHeader.slice(7);
  jwt.verify(token, getKey, { algorithms: ['RS256'] }, (err,
    decoded) => {
    if (err) {
      res.status(401).json({ error: 'Invalid token' });
      return;
    }
  });
}

```

```

    }
    (req as Request & { tenantId: string }).tenantId = (decoded
      as jwt.JwtPayload).tenantId as string;
    next();
  });
}

```



Step 5: Run tests and confirm pass

```
npx jest tests/middleware/emporixJwt.test.ts --no-coverage
```

Expected: 2 passing.



Step 6: Commit

```

git add src/middleware/ tests/middleware/
git commit -m "feat: rate limiter and Emporix JWT middleware"

```

Task 11: Punchout Setup Routes

Files: - Create: src/routes/punchout.ts - Create: tests/routes/punchout.test.ts



Step 1: Write failing tests — tests/routes/punchout.test.ts

```

import request from 'supertest';
import nock from 'nock';
import { app } from '../../src/index';

// Note: install supertest: npm install --save-dev supertest
// @types/supertest

const VALID_CXML_SETUP = `<?xml version="1.0" encoding="UTF-8"?>
<cXML payloadID="test-1" timestamp="2026-05-11T10:00:00Z">
  <Header>
    <From><Credential domain="NetworkId"><Identity>buyer-org-001</Identity></Credential></From>
    <To><Credential domain="NetworkId"><Identity>supplier</Identity></Credential></To>
    <Sender>
      <Credential domain="NetworkId">
        <Identity>buyer-org-001</Identity>
        <SharedSecret>s3cr3t</SharedSecret>
      </Credential>
      <UserAgent>TestBuyer</UserAgent>
    </Sender>
  </Header>
  <Request deploymentMode="production">

```

```

    <PunchOutSetupRequest operation="create">
      <BuyerCookie>cookie-001</BuyerCookie>
      <BrowserFormPost><URL>https://buyer.example.com/return</
        URL></BrowserFormPost>
    </PunchOutSetupRequest>
  </Request>
</cXML>`;

```

```

describe('POST /punchout/cxml/setup', () => {
  it('returns 200 cXML setup response for valid request', async
    () => {
      // Config must be mocked/seeded for this test – see
      // integration test for full flow
      const res = await request(app)
        .post('/punchout/cxml/setup')
        .set('Content-Type', 'text/xml')
        .send(VALID_CXML_SETUP);
      // Accepts 200 (config found) or 503 (config not seeded in
      // unit test)
      expect([200, 503]).toContain(res.status);
    });

  it('returns 400 for malformed XML', async () => {
    const res = await request(app)
      .post('/punchout/cxml/setup')
      .set('Content-Type', 'text/xml')
      .send('<broken>');
    expect(res.status).toBe(400);
  });
});

describe('POST /punchout/oci/setup', () => {
  it('returns 400 when HOOK_URL missing', async () => {
    const res = await request(app)
      .post('/punchout/oci/setup')
      .type('form')
      .send({ USERNAME: 'buyer', PASSWORD: 'pass' });
    expect(res.status).toBe(400);
  });
});

```



Step 2: Add supertest dev dependency

```
npm install --save-dev supertest @types/supertest
```



Step 3: Run to confirm failure

```
npx jest tests/routes/punchout.test.ts --no-coverage
```

Expected: FAIL — routes not mounted.



Step 4: Create src/routes/punchout.ts

```
import { Router, Request, Response } from 'express';
import { v4 as uuidv4 } from 'uuid';
import { parsePunchOutSetupRequest } from '../protocols/cxml/
  parser';
import { buildSetupResponse, buildOrderMessage } from '../
  protocols/cxml/generator';
import { parseOciSetupRequest } from '../protocols/oci/parser';
import { buildOciReturnFields } from '../protocols/oci/generator';
import { SessionStore } from '../session/store';
import { getConfig } from '../admin/configStore';
import { verifySecret } from '../crypto';
import { config as appConfig } from '../config';
import type { CxmlCartItem } from '../protocols/cxml/types';
import type { OciReturnItem } from '../protocols/oci/types';
import type { EmporixCart } from '../emporix/types';
import axios from 'axios';

const store = new SessionStore(appConfig.redis);

export function createPunchoutRouter(tenantId: string): Router {
  const router = Router();

  router.post('/cxml/setup', async (req: Request, res: Response)
    => {
    const xmlBody = req.body as string;
    let parsed;
    try {
      parsed = parsePunchOutSetupRequest(xmlBody);
    } catch (err) {
      const msg = err instanceof Error ? err.message : 'Parse
        error';
      res.status(400).type('text/xml').send(buildCxmlError(400,
        msg));
      return;
    }

    const cfg = await getConfig(tenantId, '').catch(() => null);
    if (!cfg || !cfg.xmlEnabled) {
      res.status(503).type('text/xml').send(buildCxmlError(503,
        'Punchout not configured'));
      return;
    }

    const secretValid = await verifySecret(parsed.sharedSecret,
      cfg.sharedSecretHash);
    if (!secretValid) {
      res.status(401).type('text/xml').send(buildCxmlError(401,
        'Invalid credentials'));
      return;
    }
  })
}
```

```

const mapping = cfg.buyerMappings.find(m => m.buyerOrgId ===
  parsed.buyerOrgId);
const customerGroupId = mapping?.customerGroupId ?? '';

const sessionId = uuidv4();
const token = uuidv4();
await store.saveSession({
  sessionId,
  protocol: 'cxml',
  buyerOrgId: parsed.buyerOrgId,
  customerGroupId,
  browserFormPostUrl: parsed.browserFormPostUrl,
  buyerCookie: parsed.buyerCookie,
  emporixCartId: null,
  createdAt: new Date().toISOString(),
}, appConfig.sessionTtlSeconds);
await store.saveToken(token, sessionId,
  appConfig.tokenTtlSeconds);

const startPageUrl = `${appConfig.pluginHost}/session/${token}
  `;
res.status(200).type('text/
  xml').send(buildSetupResponse(startPageUrl));
});

router.post('/oci/setup', async (req: Request, res: Response)
  => {
  let parsed;
  try {
    parsed = parseOciSetupRequest(req.body as Record<string,
      string>);
  } catch (err) {
    const msg = err instanceof Error ? err.message : 'Parse
      error';
    res.status(400).json({ error: msg });
    return;
  }

  const cfg = await getConfig(tenantId, '').catch(() => null);
  if (!cfg || !cfg.ociEnabled) {
    res.status(503).json({ error: 'OCI punchout not
      configured' });
    return;
  }

  const secretValid = await verifySecret(parsed.password,
    cfg.sharedSecretHash);
  if (!secretValid) {
    res.status(401).json({ error: 'Invalid credentials' });
    return;
  }

  const mapping = cfg.buyerMappings.find(m => m.buyerOrgId ===
    parsed.username);

```

```

const customerGroupId = mapping?.customerGroupId ?? '';

const sessionId = uuidv4();
const token = uuidv4();
await store.saveSession({
  sessionId,
  protocol: 'oci',
  buyerOrgId: parsed.username,
  customerGroupId,
  browserFormPostUrl: parsed.hookUrl,
  buyerCookie: '',
  emporixCartId: null,
  createdAt: new Date().toISOString(),
}, appConfig.sessionTtlSeconds);
await store.saveToken(token, sessionId,
  appConfig.tokenTtlSeconds);

res.redirect(`${appConfig.pluginHost}/session/${token}`);
});

router.post('/return', async (req: Request, res: Response) => {
  const { cartId, sessionId } = req.body as { cartId?: string;
    sessionId?: string };
  const session = sessionId ? await
    store.getSession(sessionId) : null;
  if (!session) {
    res.status(410).send(expiredPage());
    return;
  }

  const cfg = await getConfig(tenantId, '').catch(() => null);
  if (!cfg) {
    res.status(503).send(expiredPage());
    return;
  }

  let cart: EmporixCart;
  try {
    const cartRes = await axios.get<EmporixCart>(
      `${appConfig.emporix.apiBase}/cart/${tenantId}/carts/${
        cartId ?? session.emporixCartId}`,
      { timeout: appConfig.outboundTimeoutMs },
    );
    cart = cartRes.data;
  } catch {
    res.status(502).send(expiredPage());
    return;
  }

  if (session.protocol === 'cxml') {
    const items: CxmlCartItem[] = cart.items.map(i => ({
      sku: i.sku,
      name: i.name,

```

```

        quantity: i.quantity,
        unitPrice: i.price.amount,
        currency: i.price.currency,
        uom: i.uom,
    }));
    const orderMsg = buildOrderMessage(session.buyerCookie,
        cfg.operationAllowed, items, cart.currency);
    try {
        await axios.post(session.browserFormPostUrl, orderMsg, {
            headers: { 'Content-Type': 'text/xml' },
            timeout: appConfig.outboundTimeoutMs,
        });
    } catch {
        res.status(502).send(expiredPage());
        return;
    }
    res.status(200).send('<html><body>Cart transferred. You may
        close this window.</body></html>');
} else {
    const ociItems: OciReturnItem[] = cart.items.map(i => ({
        description: i.name,
        matnr: i.sku,
        quantity: i.quantity,
        unit: i.uom,
        price: i.price.amount,
        currency: i.price.currency,
        vendorMat: i.sku,
    }));
    const fields = buildOciReturnFields(ociItems,
        cfg.ociOkCode);
    const formHtml =
        buildOciFormHtml(session.browserFormPostUrl, fields);
    res.status(200).send(formHtml);
}

    await store.deleteSession(session.sessionId);
});

    return router;
}

function buildCxmlError(code: number, text: string): string {
    return `<?xml version="1.0" encoding="UTF-8"?
        ><cXML><Response><Status code="${code}" text="${text}"/></
        Response></cXML>`;
}

function expiredPage(): string {
    return '<html><body><p>Your punchout session has expired or an
        error occurred. Please return to your procurement system
        and try again.</p></body></html>';
}

```

```
function buildOciFormHtml(hookUrl: string, fields: Record<string,
string>): string {
  const inputs = Object.entries(fields)
    .map(([k, v]) => `<input type="hidden" name="${k}" value="$
    {v.replace(/"/g, '&quot;')}">`)
    .join('\n');
  return `<html><body onload="document.forms[0].submit()">
<form method="POST" action="${hookUrl}">${inputs}</form>
</body></html>`;
}
```



Step 5: Mount punchout routes in src/index.ts

```
import express from 'express';
import { config } from './config';
import { createPunchoutRouter } from './routes/punchout';
import { punchoutRateLimiter } from './middleware/rateLimiter';

const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.text({ type: 'text/xml' }));

app.get('/health', (_req, res) => res.json({ status: 'ok' }));
app.use('/punchout', punchoutRateLimiter,
  createPunchoutRouter(config.emporix.tenantId));

if (require.main === module) {
  app.listen(config.port, () => {
    console.log(`Punchout plugin listening on port ${config.port}
    `);
  });
}

export { app };
```



Step 6: Run tests and confirm pass

```
npx jest tests/routes/punchout.test.ts --no-coverage
```

Expected: 3 passing.



Step 7: Commit

```
git add src/routes/punchout.ts src/index.ts tests/routes/
punchout.test.ts
git commit -m "feat: cXML and OCI punchout setup and return
routes"
```

Task 12: Session Start Endpoint

Files: - Create: src/routes/session.ts - Create: tests/routes/session.test.ts



Step 1: Write failing tests — tests/routes/session.test.ts

```
import request from 'supertest';
import { app } from '../../src/index';
import { SessionStore } from '../../src/session/store';
import { config as AppConfig } from '../../src/config';
import nock from 'nock';

describe('GET /session/:token', () => {
  let store: SessionStore;

  beforeEach(() => {
    store = new SessionStore(AppConfig.redis);
  });

  afterEach(async () => {
    await store.disconnect();
    nock.cleanAll();
  });

  it('redirects to storefront on valid token', async () => {
    const sessionId = 'sess-redirect-test';
    await store.saveSession({
      sessionId,
      protocol: 'cxml',
      buyerOrgId: 'buyer-1',
      customerGroupId: 'cg-1',
      browserFormPostUrl: 'https://buyer.example.com/return',
      buyerCookie: 'cookie-1',
      emporixCartId: null,
      createdAt: new Date().toISOString(),
    }, 7200);
    await store.saveToken('valid-token-001', sessionId, 900);

    nock('https://api.emporix.io')
      .post(`/cart/${AppConfig.emporix.tenantId}/carts`)
      .reply(201, { cartId: 'new-cart-001' });

    const res = await request(app).get('/session/valid-token-001');
    expect(res.status).toBe(302);
  });

  it('returns 410 for expired/missing token', async () => {
    const res = await request(app).get('/session/nonexistent-token');
    expect(res.status).toBe(410);
  });
});
```

```

    expect(res.text).toContain('expired');
  });
});

```



Step 2: Run to confirm failure

```
npx jest tests/routes/session.test.ts --no-coverage
```

Expected: FAIL — /session route not mounted.



Step 3: Create src/routes/session.ts

```

import { Router, Request, Response } from 'express';
import { SessionStore } from '../session/store';
import { EmporixClient } from '../emporix/client';
import { TokenCache } from '../emporix/auth';
import { getConfig } from '../admin/configStore';
import { config as appConfig } from '../config';

export function createSessionRouter(tenantId: string): Router {
  const router = Router();
  const store = new SessionStore(appConfig.redis);

  router.get('/:token', async (req: Request, res: Response) => {
    const { token } = req.params;
    const sessionId = await store.consumeToken(token);

    if (!sessionId) {
      res.status(410).send(
        '<html><body><p>Your punchout session has expired. Please  
return to your procurement system and try again.</p></body></html>'
      );
      return;
    }

    const session = await store.getSession(sessionId);
    if (!session) {
      res.status(410).send(
        '<html><body><p>Your punchout session has expired. Please  
return to your procurement system and try again.</p></body></html>'
      );
      return;
    }

    const cfg = await getConfig(tenantId, '').catch(() => null);
    if (!cfg) {
      res.status(503).send('<html><body><p>Plugin not  
configured.</p></body></html>');
      return;
    }
  });
}

```

```

    try {
      const tokenCache = new TokenCache(
        appConfig.emporix.apiBase,
        tenantId,
        cfg.serviceAccount.clientId,
        cfg.serviceAccount.clientSecret,
      );
      const accessToken = await tokenCache.getToken();
      const emporixClient = new
        EmporixClient(appConfig.emporix.apiBase, tenantId,
          appConfig.outboundTimeoutMs);
      const cartId = await
        emporixClient.createGuestCart(session.customerGroupId,
          `Bearer ${accessToken}`);
      await store.updateCartId(sessionId, cartId);

      res.cookie('punchout_session', sessionId, { httpOnly: true,
        sameSite: 'lax', maxAge: appConfig.sessionTtlSeconds *
          1000 });
      res.redirect(`${cfg.storefrontBaseUrl}?cartId=${cartId}`);
    } catch {
      res.status(502).send('<html><body><p>Failed to initialize
        cart. Please return to your procurement system and try
        again.</p></body></html>');
    }
  });

  return router;
}

```



Step 4: Mount session route in src/index.ts

Add after the punchout route:

```

import { createSessionRouter } from './routes/session';
// ...
app.use('/session', createSessionRouter(config.emporix.tenantId));

```



Step 5: Run tests and confirm pass

```
npx jest tests/routes/session.test.ts --no-coverage
```

Expected: 2 passing.



Step 6: Commit


```
git add src/routes/session.ts src/index.ts tests/routes/
      session.test.ts
git commit -m
      "feat: session start endpoint – token validation and guest
      cart creation"
```

Task 13: Admin API Routes

Files: - Create: src/admin/router.ts - Create: tests/routes/admin.test.ts



Step 1: Write failing tests — tests/routes/admin.test.ts

```
import request from 'supertest';
import { app } from '../../src/index';

describe('Admin routes', () => {
  it('returns 401 on GET /admin/config without token', async ()
    => {
    const res = await request(app).get('/admin/config');
    expect(res.status).toBe(401);
  });

  it('returns 401 on POST /admin/config without token', async ()
    => {
    const res = await request(app).post('/admin/config').send({});
    expect(res.status).toBe(401);
  });

  it('returns 401 on GET /admin/buyers without token', async ()
    => {
    const res = await request(app).get('/admin/buyers');
    expect(res.status).toBe(401);
  });
});
```



Step 2: Run to confirm failure

```
npx jest tests/routes/admin.test.ts --no-coverage
```

Expected: FAIL — admin routes not mounted (404 not 401).



Step 3: Create src/admin/router.ts

```
import { Router, Request, Response } from 'express';
import { emporixJwtMiddleware } from '../../middleware/emporixJwt';
import { getConfig, saveConfig, PluginConfig } from './
  configStore';
import { EmporixClient } from '../../emporix/client';
import { TokenCache } from '../../emporix/auth';
```

```

import { config as appConfig } from '../config';

export function createAdminRouter(tenantId: string): Router {
  const router = Router();
  router.use(emporixJwtMiddleware);
  router.use((_req, res, next) => {
    res.setHeader('Content-Security-Policy', "default-src 'self';
      frame-ancestors 'self' https://admin.emporix.io");
    next();
  });

  router.get('/config', async (req: Request, res: Response) => {
    const cfg = await getConfig(tenantId, '').catch(() => null);
    if (!cfg) { res.json({}); return; }
    const { serviceAccount, sharedSecretHash: _omit, ...safe } =
      cfg;
    res.json({ ...safe, serviceAccount: { clientId:
      serviceAccount.clientId, clientSecret: '***' } });
  });

  router.post('/config', async (req: Request, res: Response) => {
    try {
      await saveConfig(tenantId, req.body as Parameters<typeof
        saveConfig>[1], '');
      res.json({ ok: true });
    } catch (err) {
      res.status(500).json({ error: 'Failed to save config' });
    }
  });

  router.get('/buyers', async (_req: Request, res: Response) => {
    const cfg = await getConfig(tenantId, '').catch(() => null);
    res.json(cfg?.buyerMappings ?? []);
  });

  router.post('/buyers', async (req: Request, res: Response) => {
    const cfg = await getConfig(tenantId, '').catch(() => null);
    if (!cfg) { res.status(404).json({ error: 'Config not
      found' }); return; }
    const { buyerOrgId, customerGroupId } = req.body as
      { buyerOrgId: string; customerGroupId: string };
    const existing = cfg.buyerMappings.findIndex(m =>
      m.buyerOrgId === buyerOrgId);
    if (existing >= 0)
      cfg.buyerMappings[existing].customerGroupId =
        customerGroupId;
    else cfg.buyerMappings.push({ buyerOrgId, customerGroupId });
    await saveConfig(tenantId, cfg, '');
    res.json(cfg.buyerMappings);
  });

  router.get('/customer-groups', async (_req: Request, res:
    Response) => {
    const cfg = await getConfig(tenantId, '').catch(() => null);

```

```

    if (!cfg) { res.json([]); return; }
    try {
      const tokenCache = new
        TokenCache(appConfig.emporix.apiBase, tenantId,
          cfg.serviceAccount.clientId,
          cfg.serviceAccount.clientSecret);
      const token = await tokenCache.getToken();
      const client = new EmporixClient(appConfig.emporix.apiBase,
        tenantId, appConfig.outboundTimeoutMs);
      const groups = await client.getCustomerGroups(`Bearer $
        {token}`);
      res.json(groups);
    } catch {
      res.status(502).json({ error: 'Failed to fetch customer
        groups' });
    }
  });

  return router;
}

```



Step 4: Mount admin router in src/index.ts

```

import { createAdminRouter } from './admin/router';
// ...
app.use('/admin', createAdminRouter(config.emporix.tenantId));

```



Step 5: Run tests and confirm pass

```
npx jest tests/routes/admin.test.ts --no-coverage
```

Expected: 3 passing.



Step 6: Commit

```

git add src/admin/router.ts src/index.ts tests/routes/
  admin.test.ts
git commit -m "feat: admin API routes with JWT protection"

```

Task 14: Punchout Widget

Files: - Create: src/routes/widget.ts



Step 1: Create src/routes/widget.ts

```

import { Router, Request, Response } from 'express';
import { config as appConfig } from '../config';

```

```

export function createWidgetRouter(): Router {
  const router = Router();

  router.get('/punchout-widget.js', (_req: Request, res:
    Response) => {
    res.type('application/javascript').send(`
(function() {
  function getCookie(name) {
    var match = document.cookie.match(new RegExp('(^[ ])' + name +
      '=[^;]+)'));
    return match ? match[2] : null;
  }
  var sessionId = getCookie('punchout_session');
  if (!sessionId) return;
  var btn = document.createElement('button');
  btn.textContent = 'Return Cart to Procurement System';
  btn.style.cssText = 'position:fixed;bottom:20px;right:20px;z-
    index:9999;padding:12px
    20px;background:#0066cc;color:#fff;border:none;border-
    radius:4px;cursor:pointer;font-size:14px;';
  btn.addEventListener('click', function() {
    var cartId = new
      URLSearchParams(window.location.search).get('cartId') ||
      '';
    var form = document.createElement('form');
    form.method = 'POST';
    form.action = '${appConfig.pluginHost}/punchout/return';
    ['cartId', 'sessionId'].forEach(function(key) {
      var input = document.createElement('input');
      input.type = 'hidden';
      input.name = key;
      input.value = key === 'cartId' ? cartId : sessionId;
      form.appendChild(input);
    });
    document.body.appendChild(form);
    form.submit();
  });
  document.body.appendChild(btn);
})();
`);
});

  return router;
}

```



Step 2: Mount widget route in src/index.ts

```

import { createWidgetRouter } from './routes/widget';
// ...
app.use('/', createWidgetRouter());

```



Step 3: Commit

```
git add src/routes/widget.ts src/index.ts
git commit -m "feat: punchout return cart widget JS snippet"
```

Task 15: Admin UI (React + Vite)

Files: - Create: admin-ui/package.json - Create: admin-ui/vite.config.ts - Create: admin-ui/index.html - Create: admin-ui/src/main.tsx - Create: admin-ui/src/App.tsx - Create: admin-ui/src/api.ts - Create: admin-ui/src/screens/Credentials.tsx - Create: admin-ui/src/screens/BuyerMappings.tsx - Create: admin-ui/src/screens/ProtocolSettings.tsx - Create: admin-ui/src/screens/ConnectionTest.tsx



Step 1: Create admin-ui/package.json

```
{
  "name": "emporix-punchout-admin-ui",
  "private": true,
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  "devDependencies": {
    "@types/react": "^18.2.48",
    "@types/react-dom": "^18.2.18",
    "@vitejs/plugin-react": "^4.2.1",
    "typescript": "^5.3.3",
    "vite": "^5.0.12"
  }
}
```



Step 2: Create admin-ui/vite.config.ts

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  build: { outDir: '../src/admin-ui-dist', emptyOutDir: true },
  base: '/admin-ui/',
});
```



Step 3: Create admin-ui/index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
      scale=1.0" />
    <title>Punchout Plugin Admin</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```



Step 4: Create admin-ui/src/api.ts

```
let jwtToken = '';

export function setToken(token: string) {
  jwtToken = token;
}

async function apiFetch<T>(path: string, opts?: RequestInit):
  Promise<T> {
  const res = await fetch(`/admin${path}`, {
    ...opts,
    headers: { 'Content-Type': 'application/json', Authorization:
      `Bearer ${jwtToken}`, ...opts?.headers },
  });
  if (!res.ok) throw new Error(`API error ${res.status}`);
  return res.json() as Promise<T>;
}

export const api = {
  getConfig: () => apiFetch('/config'),
  saveConfig: (body: unknown) => apiFetch('/config', { method:
    'POST', body: JSON.stringify(body) }),
  getBuyers: () => apiFetch('/buyers'),
  saveBuyer: (body: unknown) => apiFetch('/buyers', { method:
    'POST', body: JSON.stringify(body) }),
  getCustomerGroups: () => apiFetch('/customer-groups'),
  testConnection: (body: unknown) => apiFetch('/test', { method:
    'POST', body: JSON.stringify(body) }),
};
```



Step 5: Create admin-ui/src/main.tsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
```

```
import App from './App';
import { setToken } from './api';

const hash = window.location.hash.slice(1);
const params = new URLSearchParams(hash);
const token = params.get('token') ?? '';
setToken(token);

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode><App /></React.StrictMode>
);
```



Step 6: Create admin-ui/src/App.tsx

```
import React, { useState } from 'react';
import Credentials from './screens/Credentials';
import BuyerMappings from './screens/BuyerMappings';
import ProtocolSettings from './screens/ProtocolSettings';
import ConnectionTest from './screens/ConnectionTest';

const TABS = ['Credentials', 'Buyer Mappings', 'Protocol
Settings', 'Connection Test'] as const;
type Tab = typeof TABS[number];

export default function App() {
  const [tab, setTab] = useState<Tab>('Credentials');
  return (
    <div style={{ fontFamily: 'sans-serif', maxWidth: 800, margin:
'0 auto', padding: 24 }}>
      <h1 style={{ fontSize: 20, marginBottom: 16 }}>Emporix
Punchout Plugin</h1>
      <div style={{ display: 'flex', gap: 8, marginBottom: 24,
borderBottom: '1px solid #ddd' }}>
        {TABS.map(t => (
          <button key={t} onClick={() => setTab(t)}
            style={{ padding: '8px 16px', border: 'none',
background: tab === t ? '#0066cc' : 'transparent',
              color: tab === t ? '#fff' : '#333', cursor:
'pointer', borderRadius: '4px 4px 0 0' }}>
              {t}
            </button>
          </div>
        </div>
        {tab === 'Credentials' && <Credentials />}
        {tab === 'Buyer Mappings' && <BuyerMappings />}
        {tab === 'Protocol Settings' && <ProtocolSettings />}
        {tab === 'Connection Test' && <ConnectionTest />}
      </div>
    </div>
  );
}
```



Step 7: Create admin-ui/src/screens/Credentials.tsx

```
import React, { useEffect, useState } from 'react';
import { api } from '../api';

interface CredentialsForm {
  sharedSecret: string;
  clientId: string;
  clientSecret: string;
  storefrontBaseUrl: string;
}

export default function Credentials() {
  const [form, setForm] =
    useState<CredentialsForm>({ sharedSecret: '', clientId: '',
    clientSecret: '', storefrontBaseUrl: '' });
  const [status, setStatus] = useState('');

  useEffect(() => {
    api.getConfig().then((cfg: unknown) => {
      const c = cfg as Record<string, unknown>;
      setForm({
        sharedSecret: '',
        clientId: String((c.serviceAccount as Record<string,
string>)?.clientId ?? ''),
        clientSecret: '',
        storefrontBaseUrl: String(c.storefrontBaseUrl ?? ''),
      });
    }).catch(() => {});
  }, []);

  const handleSave = async (e: React.FormEvent) => {
    e.preventDefault();
    setStatus('Saving...');
    try {
      await api.saveConfig({
        sharedSecret: form.sharedSecret || undefined,
        serviceAccount: { clientId: form.clientId, clientSecret:
form.clientSecret },
        storefrontBaseUrl: form.storefrontBaseUrl,
      });
      setStatus('Saved.');
```

```
    } catch {
      setStatus('Error saving. Check console.');
```

```
    }
  };

  return (
    <form onSubmit={handleSave} style={{ display: 'flex',
flexDirection: 'column', gap: 12 }}>
      <label>Shared Secret (leave blank to keep existing)<br />
        <input type="password" value={form.sharedSecret}
onChange={e => setForm({ ...form, sharedSecret: e.target.value })}
style={{ width: '100%', padding: 8 }} />
    </form>
  );
}
```



```

    </label>
    <label>Emporix Service Account Client ID<br />
      <input value={form.clientId} onChange={e =>
setForm({ ...form, clientId: e.target.value })} style={{ width:
'100%', padding: 8 }} />
    </label>
    <label>Emporix Service Account Client Secret<br />
      <input type="password" value={form.clientSecret}
onChange={e => setForm({ ...form, clientSecret: e.target.value })}
style={{ width: '100%', padding: 8 }} />
    </label>
    <label>Storefront Base URL<br />
      <input value={form.storefrontBaseUrl} onChange={e =>
setForm({ ...form, storefrontBaseUrl: e.target.value })}
style={{ width: '100%', padding: 8 }} />
    </label>
    <button type="submit" style={{ padding: '10px 20px',
background: '#0066cc', color: '#fff', border: 'none',
borderRadius: 4, cursor: 'pointer' }}>Save</button>
    {status && <p>{status}</p>}
  </form>
);
}

```



Step 8: Create admin-ui/src/screens/BuyerMappings.tsx

```

import React, { useEffect, useState } from 'react';
import { api } from '../api';

interface Mapping { buyerOrgId: string; customerGroupId: string; }
interface CustomerGroup { id: string; name: string; }

export default function BuyerMappings() {
  const [mappings, setMappings] = useState<Mapping[]>([]);
  const [groups, setGroups] = useState<CustomerGroup[]>([]);
  const [newBuyerOrgId, setNewBuyerOrgId] = useState('');
  const [newGroupId, setNewGroupId] = useState('');
  const [status, setStatus] = useState('');

  useEffect(() => {
    Promise.all([api.getBuyers(),
api.getCustomerGroups()]).then(([b, g]) => {
      setMappings(b as Mapping[]);
      setGroups(g as CustomerGroup[]);
    }).catch(() => {});
  }, []);

  const handleAdd = async (e: React.FormEvent) => {
    e.preventDefault();
    setStatus('Saving...');
    try {
      const updated = await api.saveBuyer({ buyerOrgId:
newBuyerOrgId, customerGroupId: newGroupId });
    }
  }
}

```

```

        setMappings(updated as Mapping[]);
        setNewBuyerOrgId('');
        setNewGroupId('');
        setStatus('Saved.');
```

} catch { setStatus('Error saving.');

```

    };

    return (
      <div>
        <table style={{ width: '100%', borderCollapse: 'collapse',
marginBottom: 24 }}>
          <thead><tr><th style={{ textAlign: 'left', padding: 8,
borderBottom: '1px solid #ddd' }}>Buyer Org ID</th><th
style={{ textAlign: 'left', padding: 8, borderBottom: '1px solid
#ddd' }}>Customer Group</th></tr></thead>
          <tbody>{mappings.map(m => (
            <tr key={m.buyerOrgId}><td style={{ padding: 8 }}
>{m.buyerOrgId}</td><td style={{ padding: 8 }}>{groups.find(g =>
g.id === m.customerGroupId)?.name ?? m.customerGroupId}</td></tr>
          )}</tbody>
        </table>
        <form onSubmit={handleAdd} style={{ display: 'flex', gap: 8,
alignItems: 'flex-end' }}>
          <label>Buyer Org ID<br /><input value={newBuyerOrgId}
onChange={e => setNewBuyerOrgId(e.target.value)} style={{ padding:
8 }} /></label>
          <label>Customer Group<br />
            <select value={newGroupId} onChange={e =>
setNewGroupId(e.target.value)} style={{ padding: 8 }}>
              <option value="">Select...</option>
              {groups.map(g => <option key={g.id} value={g.id}
>{g.name}</option>)}
            </select>
          </label>
          <button type="submit" style={{ padding: '8px 16px',
background: '#0066cc', color: '#fff', border: 'none',
borderRadius: 4, cursor: 'pointer' }}>Add</button>
        </form>
        {status} && <p>{status}</p>
      </div>
    );
  }
}

```



Step 9: Create admin-ui/src/screens/ProtocolSettings.tsx

```

import React, { useEffect, useState } from 'react';
import { api } from '../api';

interface Settings { cxmlEnabled: boolean; ociEnabled: boolean;
operationAllowed: string; ociOkCode: string; }

export default function ProtocolSettings() {
  const [settings, setSettings] =

```

```

useState<Settings>({ cxmlEnabled: true, ociEnabled: true,
operationAllowed: 'edit', ociOkCode: 'ADDFROMCATALOG' }));
const [status, setStatus] = useState('');

useEffect(() => {
  api.getConfig().then((cfg: unknown) => {
    const c = cfg as Settings;
    setSettings({ cxmlEnabled: c.cxmlEnabled ?? true,
ociEnabled: c.ociEnabled ?? true, operationAllowed:
c.operationAllowed ?? 'edit', ociOkCode: c.ociOkCode ??
'ADDFROMCATALOG' });
  }).catch(() => {});
}, []);

const handleSave = async (e: React.FormEvent) => {
  e.preventDefault();
  setStatus('Saving...');
  try { await api.saveConfig(settings); setStatus('Saved.')}
catch { setStatus('Error.')}
};

return (
  <form onSubmit={handleSave} style={{ display: 'flex',
flexDirection: 'column', gap: 16 }}>
    <label><input type="checkbox" checked={settings.cxmlEnabled}
onChange={e => setSettings({ ...settings, cxmlEnabled:
e.target.checked })} /> Enable cXML</label>
    <label><input type="checkbox" checked={settings.ociEnabled}
onChange={e => setSettings({ ...settings, ociEnabled:
e.target.checked })} /> Enable OCI</label>
    <label>cXML operationAllowed<br />
    <select value={settings.operationAllowed} onChange={e =>
setSettings({ ...settings, operationAllowed: e.target.value })}
style={{ padding: 8 }}>
      {[ 'create', 'edit', 'inspect' ].map(v => <option key={v}
value={v}>{v}</option>)}
    </select>
    </label>
    <label>OCI ~OkCode<br />
    <select value={settings.ociOkCode} onChange={e =>
setSettings({ ...settings, ociOkCode: e.target.value })}
style={{ padding: 8 }}>
      <option value="ADDFROMCATALOG">ADDFROMCATALOG</option>
      <option value="SOURCING">SOURCING</option>
    </select>
    </label>
    <button type="submit" style={{ padding: '10px 20px',
background: '#0066cc', color: '#fff', border: 'none',
borderRadius: 4, cursor: 'pointer' }}>Save</button>
    {status && <p>{status}</p>}
  </form>
);
}

```



Step 10: Create admin-ui/src/screens/ConnectionTest.tsx

```
import React, { useState } from 'react';

const SAMPLE_CXML = `<?xml version="1.0" encoding="UTF-8"?>
<cXML payloadID="test-001" timestamp="${new
Date().toISOString()}">
  <Header>
    <From><Credential domain="NetworkId"><Identity>test-buyer-
org</Identity></Credential></From>
    <To><Credential domain="NetworkId"><Identity>supplier</
Identity></Credential></To>
    <Sender>
      <Credential domain="NetworkId">
        <Identity>test-buyer-org</Identity>
        <SharedSecret>YOUR_SHARED_SECRET</SharedSecret>
      </Credential>
      <UserAgent>ConnectionTest</UserAgent>
    </Sender>
  </Header>
  <Request deploymentMode="test">
    <PunchOutSetupRequest operation="create">
      <BuyerCookie>test-cookie-001</BuyerCookie>
      <BrowserFormPost><URL>https://example.com/return</URL></
BrowserFormPost>
    </PunchOutSetupRequest>
  </Request>
</cXML>`;

export default function ConnectionTest() {
  const [result, setResult] = useState('');
  const [loading, setLoading] = useState(false);

  const runTest = async () => {
    setLoading(true);
    setResult('');
    try {
      const res = await fetch('/punchout/cxml/setup', {
        method: 'POST',
        headers: { 'Content-Type': 'text/xml' },
        body: SAMPLE_CXML,
      });
      const text = await res.text();
      setResult(`HTTP ${res.status}\n\n${text}`);
    } catch (err) {
      setResult(`Error: ${err}`);
    } finally {
      setLoading(false);
    }
  };

  return (
```

```

    <div>
      <p style={{ marginBottom: 16 }}>Sends a simulated cXML
      PunchOutSetupRequest to verify credentials and config. Replace
      YOUR_SHARED_SECRET in the payload with your configured shared
      secret before testing.</p>
      <button onClick={runTest} disabled={loading}
        style={{ padding: '10px 20px', background: '#0066cc',
        color: 'fff', border: 'none', borderRadius: 4, cursor:
        'pointer' }}>
        {loading ? 'Testing...' : 'Run Connection Test'}
      </button>
      {result && <pre style={{ marginTop: 16, padding: 16,
      background: '#f5f5f5', borderRadius: 4, overflow: 'auto',
      fontSize: 13 }}>{result}</pre>}
    </div>
  );
}

```



Step 11: Build admin UI and verify

```
cd admin-ui && npm install && npm run build && cd ..
```

Expected: bundle output in src/admin-ui-dist/.



Step 12: Serve admin UI from Express — add to src/index.ts

```

import path from 'path';
// ...
app.use('/admin-ui', express.static(path.join(__dirname, '../src/
  admin-ui-dist')));

```



Step 13: Commit

```

git add admin-ui/ src/index.ts
git commit -m "feat: admin UI – React app with credentials, buyer
  mappings, protocol settings, connection test"

```

Task 16: Plugin Manifest and Docker

Files: - Create: plugin.json - Create: Dockerfile - Create: docker-compose.yml - Create: .env.example



Step 1: Create plugin.json

```

{
  "id": "emporix-punchout-plugin",
  "name": "Punchout Connector",

```

```

"version": "1.0.0",
"description": "Enables cXML and OCI punchout for enterprise
    procurement system buyers.",
"vendor": "Your Company",
"category": "B2B",
"endpoint": "https://{plugin_host}",
"adminUI": {
    "extensionPoint": "settings",
    "url": "https://{plugin_host}/admin-ui/index.html"
},
"endpoints": [
    { "path": "/punchout/cxml/setup", "method": "POST",
      "description": "cXML PunchOutSetupRequest endpoint" },
    { "path": "/punchout/oci/setup", "method": "POST",
      "description": "OCI setup form POST endpoint" },
    { "path": "/punchout/return", "method": "POST",
      "description": "Cart return endpoint" },
    { "path": "/punchout-widget.js", "method": "GET",
      "description": "Storefront punchout widget" }
],
"requiredScopes": [
    "cart.cart_manage",
    "customer-group.customergroup_view",
    "configuration.configuration_manage"
],
"configSchema": {
    "type": "object",
    "properties": {
        "PLUGIN_HOST": { "type": "string", "description": "Public
            URL where this plugin is hosted" },
        "REDIS_HOST": { "type": "string", "description": "Redis
            hostname" },
        "EMPORIX_TENANT_ID": { "type": "string", "description":
            "Emporix tenant ID" },
        "AES_KEY": { "type": "string", "description": "32-character
            AES-256 encryption key for secrets at rest" }
    },
    "required": ["PLUGIN_HOST", "EMPORIX_TENANT_ID", "AES_KEY"]
}
}

```



Step 2: Create Dockerfile

```

FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
RUN npm run build

```

```

FROM node:20-alpine
WORKDIR /app

```

```
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/src/admin-ui-dist ./src/admin-ui-dist
EXPOSE 3000
CMD ["node", "dist/index.js"]
```



Step 3: Create docker-compose.yml

```
version: '3.9'
services:
  plugin:
    build: .
    ports:
      - "3000:3000"
    environment:
      - PORT=3000
      - PLUGIN_HOST=${PLUGIN_HOST}
      - REDIS_HOST=redis
      - REDIS_PORT=6379
      - EMPORIX_API_BASE=${EMPORIX_API_BASE:-https://api.emporix.io}
      - EMPORIX_TENANT_ID=${EMPORIX_TENANT_ID}
      - EMPORIX_JWKS_URI=${EMPORIX_JWKS_URI:-https://api.emporix.io/.well-known/jwks.json}
      - AES_KEY=${AES_KEY}
    depends_on:
      - redis
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    restart: unless-stopped
    volumes:
      - redis_data:/data

volumes:
  redis_data:
```



Step 4: Create .env.example

```
PORT=3000
PLUGIN_HOST=https://your-plugin-domain.example.com
REDIS_HOST=localhost
REDIS_PORT=6379
EMPORIX_API_BASE=https://api.emporix.io
EMPORIX_TENANT_ID=your-tenant-id
EMPORIX_JWKS_URI=https://api.emporix.io/.well-known/jwks.json
```

```
AES_KEY=change-me-to-32-char-random-key!!
OUTBOUND_TIMEOUT_MS=5000
```



Step 5: Commit

```
git add plugin.json Dockerfile docker-compose.yml .env.example
git commit -m "feat: Emporix Marketplace plugin manifest,
    Dockerfile, and docker-compose"
```

Task 17: Integration Test — Full Punchout Flow

Files: - Create: tests/integration/punchout-flow.test.ts



Step 1: Write integration test — tests/integration/punchout-flow.test.ts

```
import request from 'supertest';
import nock from 'nock';
import { app } from '../../src/index';
import { config as appConfig } from '../../src/config';
import { hashSecret } from '../../src/crypto';
import { getConfig } from '../../src/admin/configStore';

jest.mock('../../src/admin/configStore');

const TENANT = appConfig.emporix.tenantId;
const API_BASE = appConfig.emporix.apiBase;

const SETUP_XML = (sharedSecret: string) => `<?xml version="1.0"
    encoding="UTF-8"?>
<cXML payloadID="integration-test-1"
    timestamp="2026-05-11T10:00:00Z">
  <Header>
    <From><Credential domain="NetworkId"><Identity>buyer-org-
      integration</Identity></Credential></From>
    <To><Credential domain="NetworkId"><Identity>supplier</
      Identity></Credential></To>
    <Sender>
      <Credential domain="NetworkId">
        <Identity>buyer-org-integration</Identity>
        <SharedSecret>${sharedSecret}</SharedSecret>
      </Credential>
      <UserAgent>IntegrationTest</UserAgent>
    </Sender>
  </Header>
  <Request deploymentMode="test">
    <PunchOutSetupRequest operation="create">
      <BuyerCookie>integration-cookie-001</BuyerCookie>
      <BrowserFormPost><URL>https://buyer.example.com/cxml/
        return</URL></BrowserFormPost>
```



```

    </PunchOutSetupRequest>
  </Request>
</cXML>`;

describe('Full cXML punchout flow', () => {
  const SHARED_SECRET = 'integration-test-secret';

  beforeAll(async () => {
    const hash = await hashSecret(SHARED_SECRET);
    (getConfig as jest.Mock).mockResolvedValue({
      sharedSecretHash: hash,
      serviceAccount: { clientId: 'test-client', clientSecret:
        'test-secret' },
      storefrontBaseUrl: 'https://store.example.com',
      cxmlEnabled: true,
      ociEnabled: true,
      operationAllowed: 'edit',
      ociOkCode: 'ADDFROMCATALOG',
      buyerMappings: [{ buyerOrgId: 'buyer-org-integration',
        customerGroupId: 'cg-premium' }],
    });
  });

  afterEach(() => nock.cleanAll());

  it('Step 1: POST /punchout/cxml/setup returns StartPage URL',
    async () => {
      const res = await request(app)
        .post('/punchout/cxml/setup')
        .set('Content-Type', 'text/xml')
        .send(SETUP_XML(SHARED_SECRET));

      expect(res.status).toBe(200);
      expect(res.text).toContain('PunchOutSetupResponse');
      expect(res.text).toContain('/session/');
    });

  it('Step 2: GET /session/:token with valid token creates cart
    and redirects', async () => {
    // First get a token via setup
    nock(API_BASE)
      .post(`/customerlogin/auth/anonymous/token`)
      .reply(200, { access_token: 'tok-int-001', expires_in:
        3600 });
    nock(API_BASE)
      .post(`/cart/${TENANT}/carts`)
      .reply(201, { cartId: 'cart-int-001' });

    const setupRes = await request(app)
      .post('/punchout/cxml/setup')
      .set('Content-Type', 'text/xml')
      .send(SETUP_XML(SHARED_SECRET));
  });
});

```

```

    const tokenMatch = setupRes.text.match(/\/session\/([a-f0-9-]+\/);
    expect(tokenMatch).not.toBeNull();
    const token = tokenMatch![1];

    const sessionRes = await request(app).get(`/session/${token}`);
    expect(sessionRes.status).toBe(302);
    expect(sessionRes.headers['location']).toContain('cart-int-001');
  });

it('Step 3: GET /session/:token with same token returns 410 (single-use)', async () => {
  const setupRes = await request(app)
    .post('/punchout/cxml/setup')
    .set('Content-Type', 'text/xml')
    .send(SETUP_XML(SHARED_SECRET));

  const tokenMatch = setupRes.text.match(/\/session\/([a-f0-9-]+\/);
  const token = tokenMatch![1];

  nock(API_BASE).post(`/customerlogin/auth/anonymous/token`).reply(200, { access_token: 'tok-2', expires_in: 3600 });
  nock(API_BASE).post(`/cart/${TENANT}/carts`).reply(201, { cartId: 'cart-single-use' });

  await request(app).get(`/session/${token}`);
  const secondAttempt = await request(app).get(`/session/${token}`);
  expect(secondAttempt.status).toBe(410);
});

it('Step 4: Returns 401 for invalid shared secret', async () => {
  {
    const res = await request(app)
      .post('/punchout/cxml/setup')
      .set('Content-Type', 'text/xml')
      .send(SETUP_XML('wrong-secret'));

    expect([200, 401]).toContain(res.status);
    if (res.status === 200) expect(res.text).toContain('401');
  }
});

describe('OCI punchout flow', () => {
  const SHARED_SECRET = 'integration-test-secret';

  beforeAll(async () => {
    const hash = await hashSecret(SHARED_SECRET);
    (getConfig as jest.Mock).mockResolvedValue({
      sharedSecretHash: hash,
    });
  });
});

```

```

    serviceAccount: { clientId: 'test-client', clientSecret:
      'test-secret' },
    storefrontBaseUrl: 'https://store.example.com',
    cxmlEnabled: true,
    ociEnabled: true,
    operationAllowed: 'edit',
    ociOkCode: 'ADDFROMCATALOG',
    buyerMappings: [],
  });
});

it('POST /punchout/oci/setup redirects to session URL', async
  () => {
    const res = await request(app)
      .post('/punchout/oci/setup')
      .type('form')
      .send({
        HOOK_URL: 'https://sap.example.com/oci/return',
        USERNAME: 'oci-buyer-001',
        PASSWORD: SHARED_SECRET,
        '~OkCode': 'ADDFROMCATALOG',
        '~Caller': 'SAPLMOB',
      });

    expect(res.status).toBe(302);
    expect(res.headers['location']).toContain('/session/');
  });
});

```



Step 2: Run integration tests

```
npx jest tests/integration/ --no-coverage
```

Expected: all passing.



Step 3: Run all tests

```
npx jest --no-coverage
```

Expected: all test suites passing.



Step 4: Final commit

```

git add tests/integration/
git commit -m "test: integration tests for full cXML and OCI
  punchout flows"

```

Spec Coverage Check

Spec requirement	Task
cXML PunchOutSetupRequest parsing	Task 4
cXML PunchOutSetupResponse generation	Task 5
cXML PunchOutOrderMessage generation	Task 5
OCI form field parsing	Task 6
OCI return field generation	Task 6
Redis session store (two-tier TTL)	Task 2
Single-use token (replay prevention)	Task 2, 12
Session expiry HTML error page	Task 12
Buyer org → customer group mapping	Task 11, 13
Emporix guest cart creation with customer group	Task 8, 12
Live cart fetch on return	Task 11
operationAllowed=edit in cXML return	Task 5
ADDFROMCATALOG OCI return	Task 6
Shared secret HMAC validation	Task 11
AES-256 encrypted service account credentials	Task 3, 9
bcrypt-hashed shared secret	Task 3, 9
Rate limiting (15 req/min)	Task 10
Emporix JWT middleware for admin routes	Task 10, 13
Admin config screen	Task 15
Admin buyer mappings screen	Task 15
Admin protocol settings screen	Task 15
Admin connection test screen	Task 15
Punchout widget JS snippet	Task 14
Plugin manifest (plugin.json)	Task 16
Docker deployment	Task 16
5-second outbound timeout	Task 7, 8
CSP headers on admin routes	Task 13
Integration test — full cXML flow	Task 17
Integration test — OCI flow	Task 17